Agentic Parsing: Reading CSV Files the Way Humans Do

Luisa Neubauer, Robin Sternberg, Jan Kossmann*,
Mihail Stoian, Andreas Zimmerer, Dominik Horn† Pascal Pfeil†, Varun Pandey,
Johannes Thürauf, Alexander van Renen, Andreas Kipf
University of Technology Nuremberg *Snowflake †Amazon Web Services

ABSTRACT

CSV parsing typically works as follows: infer a dialect, identify a header, and decode records. However, real-world CSV files routinely violate the structural assumptions underlying this abstraction. Metadata blocks, hierarchical headers, schema drift, footnotes, and combinations of these errors break the CSV structure and lead to semantically incorrect tables. We therefore re-envision CSV parsing as a semantic reconstruction task: recovering the relational table implied by the evidence contained within a file. To this end, we introduce *CSVStorm*, a benchmark comprising 68 CSV files with realistic errors. Unlike existing benchmarks that emphasize parser survival or partial cell overlap, CSVStorm measures exact, whole-table reconstruction against human-verified ground truths. We further explore *agentic parsing* with three architectures: full LLM reconstruction, file-specific parser generation, and a hybrid system that combines dialect inference and deterministic parsing with targeted LLM inspection and validation. On CSVStorm, the best hybrid parser reconstructs 82.6% of benchmark files perfectly, compared with 55.1% for the strongest conventional baseline (CleverCSV). In contrast, end-to-end generation remains prone to truncation, hallucination, and unintended modification.

1 INTRODUCTION

CSV files remain one of the most widely used formats for exchanging tabular data [12]. Their appeal is simple: CSV files are plain text, easy to create and inspect, and supported by almost every analytical system. Their format [1] is equally simple: rows are separated by line breaks, fields by delimiters, and quotes either escape values or allow delimiters within values. Despite their simplicity, parsing them is non-trivial, as real-world CSV files often violate this model. The ease with which CSV files can be created and modified by hand is both an advantage and a liability. Conventions that make them parseable are applied inconsistently, thus introducing errors such as comments, titles, footers, schema changes, and spreadsheet artifacts.

These errors are of two types. ① *Syntactic errors* concern how values are encoded and separated; they can be resolved by inferring delimiters, quotes, encodings, or row boundaries. ② *Semantic errors* concern what the file content represents, such as whether a row is metadata or data, whether two header rows form a hierarchy, or how to split a row when fields are separated by spaces; resolving these requires semantic reconstruction from file-internal evidence.

Most conventional CSV parsers work as follows: infer a dialect, identify a single-line header, and read the data records. This works well when the intended table is syntactically recoverable (①), but does not support cases such as two-level headers or metadata blocks

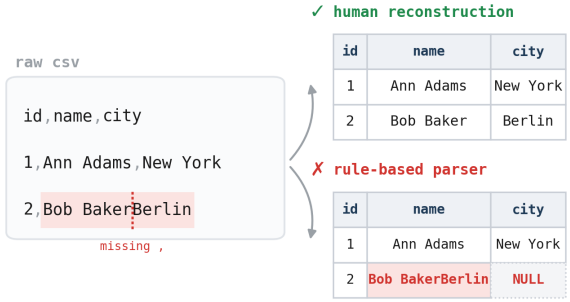


Figure 1: Missing CSV delimiter causing parser failure although the table structure is obvious to human editors.

(②). These features remain evident to humans from file inspection, but remain ambiguous to rule-based parsers. Data analysts are thus forced to waste valuable time in diagnosing and resolving ingestion errors by hand. Existing benchmarks also inherit the same assumption and test whether systems tolerate syntactic errors and system failures, but overlook semantic correctness for ingestion. These criteria are necessary yet insufficient, as a parser can produce syntactically valid tables while containing the *wrong* information. Metadata may become data, hierarchical headers may be flattened incorrectly, or values may silently shift across columns (cf. Figure 1). These failures never produce errors or crashes and, consequently, are not included in existing benchmarks.

This paper argues that CSV parsing should evolve from mere *parsing* to include *semantic reconstruction*. Our guiding principle is that any structure a human can infer from file-internal evidence should also be resolved by a parser. We call this approach *Agentic Parsing*. An agentic parser inspects the file, discovers its structure, distinguishes data based on surrounding context, validates the resulting table, and records the decisions that led from the raw artifact to the parsed relation. The goal is not to replace conventional parsers with language models, but to complement them with semantic reasoning for files that are human-readable yet parser-hostile.

To study this problem, we introduce *CSVStorm*, a benchmark for holistic CSV ingestion. CSVStorm extends prior CSV benchmarks with ② *semantic errors* found in real-world files, both individually, and in combination. Unlike benchmarks whose scores mostly reward whether a system can parse most of the cells present without crashing, CSVStorm evaluates whether the parsed result matches the intended human reconstruction.

We evaluate conventional parsers, LLM-based repair baselines, and agentic parsing architectures. Our results show that existing parsers remain effective on many syntactic cases but fail on semantic errors. Naive LLM repair can recover some cases while agentic parsing achieves the best results.

†This work is not related to the authors' positions at Amazon Web Services.
[1] https://www.rfc-editor.org/info/rfc4180/

Our contributions are as follows:

(1) We identify CSV parsing as a problem of semantic reconstruction, not merely syntactic parsing, and characterize cases where the intended table cannot be recovered from syntax alone.
(2) We introduce CSVStorm, a benchmark for evaluating holistic CSV ingestion across syntactic and semantic errors.
(3) We propose and implement three agentic parsing architectures that combine conventional parsing, semantic inspection, validation, and provenance, making CSV parsing future-proof.
(4) We provide an empirical study of parsers and LLM-based repair baselines, showing where current systems fail and why parser quality is overestimated by existing benchmarks.

2 RELATED WORK

CSV Parsing and Dialect Detection. The CSV format, the de facto standard for tabular data exchange [11, 13], lacks a universally adopted specification while its standardization, RFC 4180, is only loosely followed. CSV ingestion is thus seen as a parsing and dialect detection problem, with systems inferring delimiters, quoting rules, encodings, line endings, headers, and column types from rules, heuristics, and statistical regularities. Row patterns and type signatures provide strong signals for recovering structure from irregular files [11]. Widely used libraries such as Pandas and DuckDB robustly handle many common cases, while benchmarks such as CSVsniffer [1] and Pollock [13] have advanced the systematic evaluation of parser robustness. These approaches largely assume that the intended table can be recovered from syntax, row regularity, or local type patterns alone.

Real-World CSVs. Large-scale studies of open-data repositories further show that real-world CSV files have substantial heterogeneity in dialects, encodings, headers, and file structure [8]. The W3C tabular-data model similarly recognizes that tabular data on the Web often requires metadata to describe structure, interpretation, and processing behavior [10]. These observations suggest that reliable ingestion requires interpretation rather than syntactic parsing alone, a requirement not addressed by existing parser robustness benchmarks.

Data Cleaning, Table Understanding. Data cleaning and table-understanding systems operate on structured tables. The former detects and repairs errors, while the latter infers semantic types, learns table representations, retrieves tables, or reasons over their contents. Representative work includes Sherlock [5], GitTables [3], TASHEEH [2], studies of practical table-understanding systems [4], the TARGET retrieval benchmark [6], and evaluations of LLM reasoning over tables [14].

Agentic Data Systems. Recent LLM-powered data systems extend this scope to broader data science workflows. KramaBench [7] evaluates agents on end-to-end tasks that involve processing datasets and producing insights. In contrast, CSVStorm isolates the ingestion stage, which requires identifying table boundaries, distinguishing metadata and headers from data, and aligning columns for successful table reconstruction. These decisions produce the table representation required by downstream cleaning, typing, retrieval, and analysis.

3 CSVSTORM

Pollock [13] provides a benchmark for evaluating CSV-parser robustness. It systematically applies non-standard dialects and structural errors to a standard source file, a transformation it calls *pollution*. Next, the transformed files containing errors are parsed and the resulting output is compared to a ground truth. Although this makes failures reproducible, it leaves three gaps for evaluating agentic parsers, which must infer how polluted files should be reconstructed.

First Issue. Pollock applies only one pollution at a time. Its 2,290 files differ from the source file marginally, often only by a single altered character. This largely ignores interactions among errors that commonly co-occur in practice, since the same manual processes that introduce one error tend to introduce others as well.

Second Issue. Pollock's compound score gives little weight to local errors: a file's score is calculated by combining header-, record-, and cell-level precision, recall, and F_1. Thus, small parsing errors still produce a near-perfect score. For example, if the parser drops a whole row containing an error (e.g., a changed delimiter), the file still gets 99.6% of the maximum possible score.

Third Issue. Some of the modifications in the Pollock benchmark allow for multiple valid interpretations of the file. Consider a stray, unclosed quotation mark: *value1,"value2,value3*. The mark may be interpreted as a syntax error, as literal content, or as the opening of a quoted field whose boundary cannot be determined. Each interpretation yields a different valid reconstruction. For example, Pollock's reconstruction may preserve the quotation mark as field content, whereas an adjudicator may conclude that the intended row was the original unpolluted row, *value1,value2,value3*. However, the benchmark accepts only a single predefined solution as correct. When evaluating agentic parsers, we must therefore account for this ambiguity, especially when moving toward more semantic errors.

Summary. These limitations do not diminish Pollock's value as a *parser-robustness* benchmark. Instead, they motivate a new but complementary benchmark that preserves controlled generation while testing interacting errors, interpretation-dependent cases, and complete table correctness.

3.1 Introducing CSVStorm

We introduce *CSVStorm*, a benchmark stress-testing conventional parsers and novel agentic parsing systems on a varied set of CSV files containing realistic *syntactic* and *semantic* errors. These include large files to evaluate scalability. Its objective is not merely to test whether a system under test (SuT) returns an output without crashing, but whether it reconstructs a complete and consistent table. The current release contains 69 input files derived from one shared source table: the unpolluted source file, 62 files with a single error, and six with interacting errors. Of these six, four apply two errors and two apply three errors. The errors were motivated by prior real-world surveys [8, 13] and engineering stress tests. CSVStorm is available on GitHub and the Hugging Face Dataset Hub.

3.2 Benchmark Composition

CSVStorm defines six error categories, implemented by 28 pollution operators that produce 62 single-error files.

- **Dialect and lexical syntax** (*10 operators; 20 files*): *Syntactic.* Changes to delimiters, quotation, escaping, and record boundaries,

including mixed delimiters, non-standard quotation characters, and unescaped multiline values.

- **Record shape and alignment** (*3 operators; 14 files*): *Semantic.* Rows containing missing, additional, or displaced fields.
- **Header and schema layout** (*5 operators; 7 files*): *Semantic.* Missing, displaced, multi-row, or hierarchical headers.
- **File segmentation and table boundaries** (*3 operators; 7 files*): *Semantic.* Preambles, footnotes, comments.
- **Value and semantic interpretation** (*3 operators; 7 files*): *Semantic.* Embedded structured values, spreadsheet formulas, and heterogeneous temporal representations.
- **Operational Scale stressors** (*4 operators; 7 files*): *Syntactic.* Large row or column counts, unusually long fields, as well as degenerate tables with just one row or no payload at all.

The six categories above partition the 62 single-error files. CSVStorm additionally contains six curated combination files, each applying two or three operators drawn from different categories. These include displaced headers with preambles, multi-row headers with footnotes, and record-shape errors with mixed delimiters. The complete benchmark therefore comprises 68 polluted files and one unpolluted control.

3.3 Benchmark Construction

CSVStorm extends Pollock's [13] controlled-generation methodology. Starting from a clean source table, a pollution operator transforms a structured representation of that table and emits four artifacts: (1) the polluted CSV presented to the SuT; (2) a canonical clean CSV; (3) a JSON file recording the generated dialect and error provenance; and (4) a ground-truth bundle containing one or more accepted reconstructions. The evaluator compares the output of the SuT against the accepted reconstructions and computes diagnostic metrics. Figure 2 summarizes the benchmark construction process.

3.4 Metrics and Ground Truth

Pollock derives the expected output from the known pollution grammar [13], and we adopt this as the canonical target. In cases where more than one reconstruction may be defensible, however, we identify and include additional accepted reconstructions in the ground-truth bundle.

Pollock's header-, record-, and cell-level measures quantify partial similarity between a system's output and the expected reconstruction. CSVStorm retains these measures for error localization and characterization. In addition, we report whole-table reconstruction accuracy, which requires every row and cell to agree with the complete accepted reconstruction. We also report SuT success, defined as the proportion of benchmark files for which the SuT completes execution successfully.

Let $\mathcal{G}_i$ denote the accepted table reconstructions for instance i, and let $\hat{T}_i$ denote the system output. We define

$$\mathrm{corr}(i) = \mathbb{I}\left[\exists T \in \mathcal{G}_i : \mathrm{equiv}(\hat{T}_i, T)\right], \qquad \mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathrm{corr}(i),$$

where equiv denotes table equivalence, $\mathrm{corr}(i)$ indicates whether instance i was reconstructed correctly, and Acc is the overall exact reconstruction accuracy.

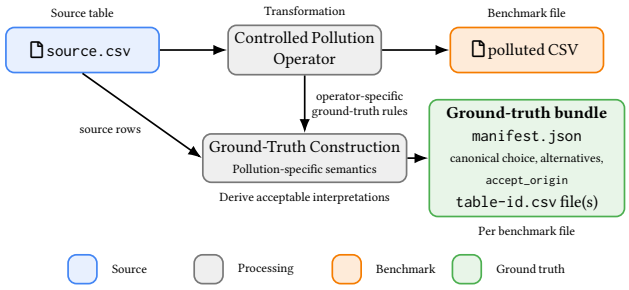


Figure 2: Benchmark and ground-truth bundle generation.

Unlike cell-level similarity, this measure grants no partial credit: a single incorrectly reconstructed row disqualifies the file.

We follow an explicit evaluation protocol in which an output is correct if it exactly matches the canonical reconstruction or any alternative approved for that file. For some files, the original source table is such an alternative. Reporting performance under this relaxed criterion is essential, as some files admit more than one acceptable result.

Scope and Limitations. CSVStorm prioritizes experimental control over source diversity. All files in the current release derive from one shared table, and the combination suite contains six manually selected operator pairings rather than an exhaustive sample of possible interactions. Performance on CSVStorm therefore establishes robustness to the included transformations but does not by itself generalize to arbitrary domains, schemas, encodings, or naturally occurring files. Expanding the benchmark to multiple source tables and empirically sampled real-world combinations is an important direction for future work.

4 AGENTIC PARSING

Our guiding principle for *agentic parsing* is: if a human can reconstruct a table encoded in a text file (e.g., a mutated CSV file) from evidence contained in the file, then an automated system should be able to do the same. Given an input file f, an agentic parser produces a relational table $\hat{\mathcal{T}}$ and records the decisions it made while interpreting and transforming the file.

4.1 Architectural Design Space

Traditional parsers use statistics and heuristics to determine the CSV file's grammar before parsing. An LLM can augment the ingestion pipeline in varying ways. It can be limited to localized decisions while a conventional parser retains control of serialization and validation, or it can regenerate the entire repaired table directly. It can also act indirectly and generate custom parser code for an individual file which then processes it entirely. Based on these design scopes, we envision the following architectures visualized in Figure 3:

(1) **Hybrid Parser**, combining dialect and layout inference, deterministic parsing, anomaly detection, and localized LLM repair. Restricting model intervention to ambiguous decisions and malformed records reduces model calls while allowing conventional parsers to handle well-formed content.
(2) **Full Agentic Parsing**, which transforms the input file into a normalized CSV in a single generation step. Maximizes model

freedom and gives the model access to the full context but requires it to reproduce the entire payload correctly.

(3) **Custom Parser Code**, in which an LLM synthesizes a file-specific parsing function from samples and applies it to the complete file. Amortizes model reasoning over the file but introduces program-validation and execution risks.

Hybrid Parser. This is our reference architecture and forms a hybrid approach. First, the CSV dialect is inferred by both a conventional rule-based sniffer (e.g., CleverCSV or DuckDB) and an LLM that examines the first lines of the input file. The resulting dialects are reconciled by generating additional candidates from combinations of their delimiters and quotation characters. Each candidate is scored by the uniformity of the resulting rows, and the highest-scoring dialect is selected. The file is then parsed with DuckDB in strict mode to expose malformed rows. Because DuckDB may overlook column-count mismatches when the final column contains a null value, a separate dialect-aware column counter serves as an additional validation step. Rows identified as erroneous are passed to the LLM together with valid examples, the detected dialect, and the corresponding parser error. The LLM either repairs them or identifies them as non-data rows to be discarded. Repaired rows are returned as JSON and merged with the successfully parsed rows to produce the final output.

Full-Agentic Parser. The full-agentic parser is implemented as an end-to-end agentic system. For each input file, the system reads the potentially malformed CSV as raw text together with a detailed prompt. The LLM is asked to return a representation with two sections: a complete, valid comma-separated CSV containing a single flattened header row, and a structured report describing alterations, exclusions, ambiguities, and validation decisions. To ensure reproducibility and control cost, requests are deterministic and locally cached. After generation, the parser validates the reconstructed CSV with pandas.

Custom Parser Code. An LLM synthesizes a file-specific parser. The model receives the first n lines of the input file, a sample of records distributed across the remainder, and the mechanically inferred dialect and schema. It then generates a Python parsing function that is executed on the complete input file to produce the final relation. Generated programs are restricted to safe standard-library modules and terminated if execution exceeds a fixed timeout.

4.2 Interactive Demo

To make agentic parsing directly accessible, we provide an interactive web demo that allows users to upload CSV files and inspect the reconstructed output produced by our agentic parsing system: https://utndatasystems.github.io/agentic-parsing.

5 EVALUATION

5.1 Systems Under Test and Metrics

We evaluate the three agentic parsing architectures against a set of widely used CSV ingestion systems. All Python-based systems were executed using Python 3.13.11. The conventional baselines include Pandas [9]; CleverCSV [11], a drop-in replacement for Python's standard `csv` module; the built-in Python CSV reader (`PyCSV`); and two DuckDB configurations: `DuckDB-Automatic`, which performs automatic dialect and schema detection, and `DuckDB-Explicit`, which parses with a predefined configuration.

The above systems can be differentiated by the information about input file structure they can access. We divide the systems into two categories, separating oracle-assisted from fully automatic parsers.
Automatic parsers: `DuckDB-Automatic`, PyCSV, CleverCSV, and the LLM-based systems receive the plain input CSV without any additional information: dialect, structure, and schema decisions must be inferred from the input alone.
Oracle-assisted parsers: `DuckDB-Explicit` receives the delimiter, quoting configuration, header and preamble information, and exact column names. Pandas receives oracle encoding, quoting, escaping, and preamble information.

We compare systems using reconstruction accuracy (the number of correctly vs. incorrectly parsed CSV files), and the original Pollock score to allow for comparison with prior work and to distinguish complete reconstruction failures from near-correct outputs. We report results on CSVStorm. Row permutations are accepted as correct provided the row content is identical.

The GitHub repository also contains the complete implementation, evaluation scripts, prompts, and model configurations used in our experiments.

5.2 Results and Findings

SuT Comparison. Table 1 reports the performance of the automatic and oracle-assisted systems.

Overall performance. The best performing hybrid, full-agentic, and custom parser code systems achieve 82.6%, 73.9%, and 49.3% accuracy, respectively. Among the baselines, CleverCSV reaches 55.1% accuracy, while oracle-assisted DuckDB parsing reaches 40.6%. Note that the input to the evaluated SuTs varied from plain files (automatic) to detailed per-file parameters (oracle-assisted), as outlined earlier.

Architecture and model effects. We tested the LLM-based approaches on the current GPT-5.6 Sol flagship (released July 2026), the smaller GPT-5.6 Luna, the older GPT-5.4 Mini, and the lightweight, open Qwen3.5 0.8B models. The reasoning level was set to medium. Accuracy generally improved with model capability, particularly for the hybrid and fully agentic approaches. The hybrid approach performed best overall, indicating that combining LLM-based inference with deterministic parsing is more effective than relying on either full-LLM reconstruction or generated parsing code alone.

Robustness versus correctness. SuT success measures the proportion of cases in which a system returns output without crashing, whereas accuracy requires exact reconstruction of the intended table. We observe that these measures diverge substantially. For instance, DuckDB Parse has 100% SuT success rate but reconstructs only 40.6% of the files, while PyCSV completes 94.2% but achieves only 10.1% accuracy. This clearly shows that execution success does not imply *correct* ingestion.

SuT success also varies widely among the LLM-based SuTs mainly due to different instruction-following capabilities. GPT-5.4 Mini illustrates this, achieving SuT success rates of 94.2% and 95.7% in hybrid and custom parser code runs, respectively, but only 53.6% when tasked with reconstructing the table directly. Qwen3.5 0.8B is the

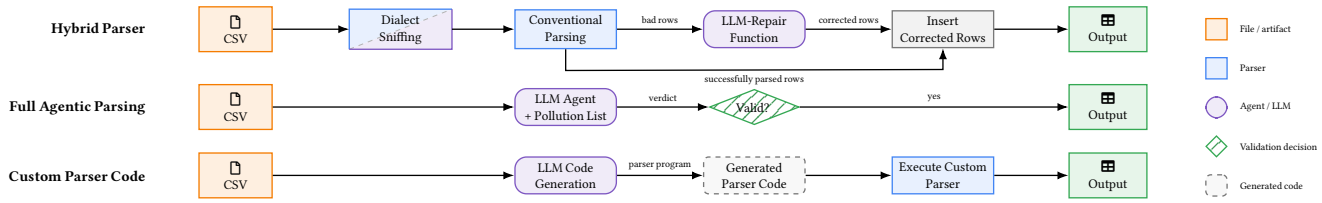


Figure 3: Three agentic CSV parsing architectures: hybrid parser, full agentic parsing, and custom parser-code generation.

	System	SuT Success [%]	Pollock Score	Accuracy [%]
Hybrid	**GPT-5.6 Sol**	**100.0**	**9.94**	**82.6**
	GPT-5.6 Luna	100.0	9.71	73.9
	GPT-5.4 Mini	94.2	9.23	63.8
	Qwen3.5 0.8B	100.0	7.53	27.5
Full Agentic	GPT-5.6 Sol	98.6	9.57	73.9
	GPT-5.4 Mini	53.6	5.21	37.7
Custom Parser Code	GPT-5.6 Sol	68.1	6.71	44.9
	GPT-5.6 Luna	79.7	7.75	49.3
	GPT-5.4 Mini	95.7	8.46	34.8
	Qwen3.5 0.8B	0.0	0.00	0.0
Baselines	CleverCSV	100.0	8.67	55.1
	DuckDB Parse*	100.0	9.27	40.6
	DuckDB Auto	98.6	8.22	34.8
	PyCSV	94.2	8.73	10.1
	Pandas*	98.6	8.06	33.3

* denotes oracle-assisted parsing with access to file-format metadata such as the dialect.

Table 1: CSVStorm results on 68 irregular and one control file. SuT Success measures non-crashing runs. Accuracy measures exact reconstruction.

extreme case, achieving 100% SuT success in the hybrid architecture but 0% in the custom parser code architecture.

The Pollock score often remains high even when exact accuracy changes drastically, directly pointing to one of the issues with the Pollock score. This means that the SuTs manage to parse the files but their content is incorrect. For example, PyCSV and CleverCSV score 8.73 and 8.67 on Pollock score, but only reconstruct 10.1% and 55.1% of files correctly. These systems often preserve much of the payload but make meaningful mistakes such as treating preambles as content, selecting the wrong header, or merging table regions that should remain distinct. Such results can be syntactically valid but logically wrong, which is exactly the type of failure CSVStorm is intended to expose.

Performance by Error Category. To categorize the challenges for individual SuTs, we break down performance by error category. Figure 4 reports the average correctness score for each system and error type. Header and schema layout remains the hardest category for conventional parsers, which score between 0 and 28.6%, because recovering displaced, multi-row, or hierarchical headers requires a semantic understanding of rows and regions within files. Both hybrid and full agentic parsers improve sharply here (100% and 85.7%), and on combination files, where every conventional parser scores zero.

Operational Scale Stressors. CleverCSV handled all seven cases, while Pandas, explicitly configured DuckDB, and DuckDB Auto each failed only on the empty file. Python's CSV reader scored lowest and also failed the unpolluted control, so its errors reflect doubled-quote handling rather than size limits. The hybrid LLM parser handled all seven cases, whereas full-agentic reconstruction failed on every large instance because it had to regenerate the complete payload.

Combination Files. Only LLM-based SuTs correctly parsed any files with multiple errors.

Summary. In summary, these results support the central claim of CSVStorm: modern CSV parsing cannot be evaluated solely based on syntax recovery. Conventional parsers remain valuable for low-level structure, but they fail sharply when correctness depends on interpreting file-internal evidence. Agentic parsing substantially improves performance on these cases, while also revealing the need for stronger validation, provenance, and multi-ground-truth evaluation. The main takeaway is that the hard cases in CSV ingestion are no longer just malformed quotes, delimiters, or row lengths. Instead, they are interpretive. Once the intended table must be inferred from context, pure rule-based parsing alone is no longer the right abstraction for CSV ingestion.

5.3 Failure Modes of Full Agentic Parsing

Naive end-to-end CSV repair proved unreliable for several reasons: (i) regenerating entire CSV files with JSON error reports creates substantial output-length pressure. Many benchmark files exceed the completion limits of smaller models, making truncation failures an inherent limitation of this design.

(ii) Full agentic parsers frequently altered the original payload. Models silently canonicalized values (e.g., converting `2` to `2.0` or replacing `\'` with `'`), and occasionally hallucinated entirely new content. For example, on `file_no_payload.csv`, the model invented a `Name,Age,Occupation` table although empty output is expected. These are generation rather than CSV dialect-detection failures.

(iii) Instruction following varied substantially across model families. Smaller Qwen models frequently failed to produce the required output schema. Even larger variants struggled to consistently emit the prescribed CSV+JSON format. In contrast, GPT-family models were considerably more reliable.

Overall, current LLMs are poorly suited for direct end-to-end CSV reconstruction. They may hallucinate data, modify existing payloads, miss subtle formatting errors, and incur high latency and token costs because the entire file must be processed. Future work should improve instruction following, structured-output reliability, and native understanding of tabular data formats.

5.4 Economics, Determinism, and Privacy

Table 2 reports the runtime and API cost of our reference architecture, the hybrid parser. Median API cost ranges from $0.0024 to $0.0177

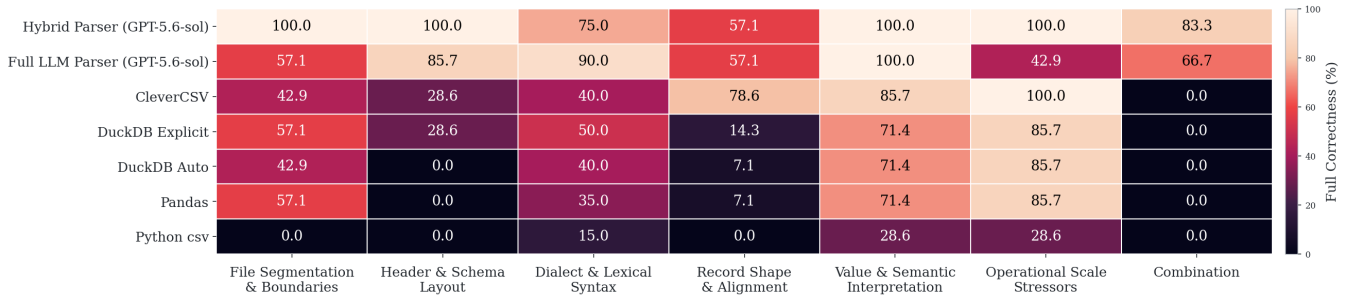


Figure 4: Correctness score by error category. Rows correspond to systems under test and columns correspond to error categories. Darker cells indicate lower performance.

Model	Acc.	Time		Cost	
	(%)↑	Median (s/file)↓	Total (s)↓	Median ($/file)↓	Total ($)↓
GPT-5.6 Sol	**82.6**	5.57	1092.54	0.0177	12.3958
GPT-5.6 Luna	73.9	4.64	563.20	0.0049	2.4736
GPT-5.4 Mini	63.8	**2.34**	269.22	**0.0024**	0.3375
Qwen3.5 0.8B	27.5	12.08	1402.32	Local	Local

Table 2: Performance, runtime, and cost of the hybrid repair approach. Local models incur no API charge.

per file across the evaluated hosted models. Unlike full-agentic reconstruction, which scales with full output size, the hybrid parser only uses the model for inference and local repair, so cost depends mainly on the number and complexity of errors rather than file size. When interpreting the measured runtimes, one should consider that hosted-model latency depends on network conditions and provider load, whereas local-model latency depends on the hardware used.

To improve reproducibility, we cache each raw model response and reuse it for identical requests. This avoids identical API calls, which may produce different results due to model-serving changes or numerical nondeterminism. Production systems could use such caching to avoid nondeterminism for frequently ingested files. Locally hosted models offer an alternative for cheaper, privacy-sensitive applications: they avoid API charges, prevent content disclosure to third-party providers, and provide greater control over the serving environment, making their results more reproducible. However, these advantages currently come at a substantial accuracy cost, as demonstrated by the low test scores of Qwen3.5 0.8B.

6 CONCLUSION

Existing CSV evaluations often conflate successful execution with correct ingestion. CSVStorm instead evaluates exact whole-table reconstruction against one or more accepted interpretations, covering both syntactic and semantic errors. Under this metric, successful parser execution is a weak proxy for correctness: PyCSV returns an output for 94.2% of the benchmark files but exactly reconstructs only 10.1%. The best evaluated hybrid parser reaches 82.6% accuracy, compared with 55.1% for the strongest evaluated classical baseline in our experiments.

Our architectural comparison also clarifies where language models are—and are not—useful in ingestion pipelines. End-to-end LLM reconstruction requires reproducing the complete payload and is therefore exposed to output-token limits, high per-file latency and cost, hallucination, and unintended value modification. File-specific parser generation avoids regenerating the payload but remains sensitive to model coding and instruction-following capabilities. In contrast, the hybrid architecture confines model reasoning to dialect, layout, and localized repair decisions while retaining deterministic parsing and validation for the remaining data. This division of labor provides the best accuracy and substantially lowers median per-file latency and cost relative to end-to-end reconstruction with an LLM.

Beyond CSV. We hypothesize that agentic parsing may also benefit other human-editable structured and semi-structured formats, such as JSON, YAML, and even HTML: Whenever a human can edit a machine-readable file without guardrails, parsers should either be flexible enough to perform intent-based parsing or confidently identify ambiguous cases and reject the input rather than succeed with a broken output.

REFERENCES

[1] W. García. Detecting CSV file dialects by table uniformity measurement and data type inference. *Data Sci.*, 7(2):55–72, 2024.

[2] M. Hameed et al. TASHEEH: Repairing Row-Structure in Raw CSV Files. In L. Tanca et al., editors, *Proceedings 27th International Conference on Extending Database Technology, EDBT 2024, Paestum, Italy, March 25 - March 28*, pages 426–439. OpenProceedings.org, 2024.

[3] M. Hulsebos et al. Gittables: A large-scale corpus of relational tables. *Proceedings of the ACM on Management of Data*, 1(1):1–17, May 2023.

[4] M. Hulsebos et al. Making table understanding work in practice, 2021. https://arxiv.org/abs/2109.05173.

[5] M. Hulsebos et al. Sherlock: A deep learning approach to semantic data type detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, pages 1500–1508, New York, NY, USA, 2019. Association for Computing Machinery.

[6] X. Ji et al. Target: Benchmarking table retrieval for generative tasks, 2025. https://arxiv.org/abs/2505.11545.

[7] E. Lai et al. Kramabench: A benchmark for ai systems on data-to-insight pipelines over data lakes, 2026. https://arxiv.org/abs/2506.06541.

[8] J. Mitlöhner et al. Characteristics of open data csv files. *2016 2nd International Conference on Open and Big Data (OBD)*, pages 72–79, 2016.

[9] J. Reback et al. pandas-dev/pandas: Pandas 1.3.2, Aug. 2021.

[10] J. Tennison et al. Model for tabular data and metadata on the web, 2015. https://www.w3.org/TR/tabular-data-model/.

[11] G. J. van den Burg et al. Wrangling messy csv files by detecting row and type patterns: Gjj van den burg et al. *Data Mining and Knowledge Discovery*, 33(6):1799–1820, 2019.

[12] A. van Renen et al. Why TPC is Not Enough: An Analysis of the Amazon Redshift Fleet. *Proc. VLDB Endow.*, 17(11):3694–3706, July 2024.

[13] G. Vitagliano et al. Pollock: A data loading benchmark. *Proc. VLDB Endow.*, 16(8):1870–1882, Apr. 2023.

[14] C. Wolff et al. How well do llms reason over tabular data, really?, 2025. https://arxiv.org/abs/2505.07453.